University of Portsmouth

School of Creative Technologies

Final Year Project

Evaluating the approachability of scientific papers towards creating a simulated particle physics scenario in C++ without prior background in the field

by

Tayler Parsons
UP2212801

Supervisor: Jahangir Uddin

April 10, 2026

Project Type: Combined

I consent to my report in this attributed format (not anonymized), subject to final approval by the Board of Examiners, being made available in the Library Dissertation Repository and/or the school digital repositories for a maximum of 10 years.

✓Yes No

Abstract

This report investigates the accessibility of scientific literature from a software engineering perspective, using the creation of a sub-atomic particle physics simulation as a case study. Using physics research, a simulation will be made that aims to realistically reproduce the behaviour of the fundamental particles, including Protons, Neutrons, Electrons. Furthermore, the simulation will attempt to represent the four fundamental interactions: the strong, weak, electromagnetic, and gravitational forces.

Through the implementation of these concepts without prior academic or professional expertise, this project aims to evaluate the translatability and level of understanding needed to take the mathematics and concepts discussed into code.

Contents

1 Introduction **2**

1.1 Project Aim 2

1.2 Subjectivity Disclaimer 2

2 Project Management & Time Keeping **3**

3 Literature Review **4**

3.1 Introduction 4

3.2 Review of existing literature 4

3.2.1 Foundational Physics 4

3.2.2 Computational Physics 5

3.2.3 Developer Experience & Accessibility 5

4 Accessibility Metric System **6**

4.0.1 Scoring Stystem 6

4.0.2 Evaluation Process 6

5 Methodology **8**

5.1 Simulation Architecture 8

5.2 Approachability Audit 8

5.2.1 Data Collection Process 8

5.2.2 Evaluation 9

5.2.3 Problem with Units & Scales 9

5.3 Fundamental Forces 10

5.3.1 Gravitational 10

References **13**

Chapter 1

Introduction

Software engineering has always been closely tied to physics and mathematics to the point where the former has become strongly associated with the latter. In some fields, such as game development which has a very large independant / solo community, this may be considered to be a barrier to entry for some. People may be dissuaded from learning how to program due to how heavy the mathematics might be, leading to the field missing out on great games made from the passion of talented people.

Resources such as research papers which are not explicitly designed with beginners-to-the-field, or game developers mystify the mathematics due to heavily verbose notation and content.

1.1 Project Aim

This project aims to evaluate and review how approachable research papers are for people who do not have a background in mathematics and physics, in creating a project.

The aim of this project is to create a plausible simulation of particle physics in C++, a project that will be heavily based in foundational physics.

1.2 Subjectivity Disclaimer

The evaluation of the accessibility and approachability of scientific papers and by extension the developer experience detailed in this report is inherently subjective, shaped by my own specific background in both particle physics (or lack thereof), and experience in programming, and interpretation of the resources used. As such, the challenges and successes documentated will vary for other developers with different backgrounds and levels of expertise in the two fields.

Chapter 2

Project Management & Time Keeping

Due to extenuating circumstances, development on this artefact and paper did experience a significant delay. I had initally spent some significant time at the beginning of the project creating the boilerplate code for the simulation, which was partially adapted from Marmalade Engine (Parsons & Bester, 2026), a modular Game Engine co-developed by myself.

Marmalade Engine was used as base due to my familiarity with the core structure, and to eliminate much of the time and effort it would take if I was to do it from scratch. Furthermore, it should be noted that the code used was primarily the rendering backend, whereas nothing relating to the development of the simulation was used.

Due to weak prioritisation of my other university commitments, and the shortcomings of development early on in the supposed project timeline, the primary development window took place across the entire month of April, 2026.

Chapter 3

Literature Review

3.1 Introduction

The purpose of a literature review is to identify existing areas of research relating to the given topic, alongside the any areas that are lacking in research literature. It also serves to provide an understanding of the topics that are intended to be discussed and expanded upon during the project.

3.2 Review of existing literature

Scientific literature regarding any subset of physics is traditionally written for those with a strong background of physics, mathematics, or research. Because of this, people outside of such disciplines, such as those in computer science including the game development fields, may be discouraged to use these resources due to the formatting, terminology or mathematic symbology that may unfamiliar to them. The result of this is developers who miss out on learning new techniques, algorithms, or optimalisation that may benefit their software or game. This literature review, and the following report evaluates foundational physics papers and their accessibility to those not discipined in physics or mathematics through the creation of a particle physics simulation in C++.

This review aims to source research within particle physics, and computer science.

3.2.1 Foundational Physics

This first category of literature consists of foundational physics papers, such as *A Standard Model Workbook* (Moore, 2024, p. .) While reports such as these serve to explain and give context to the bedrock of particle physics, they do so whilst presenting a developer experience barrier due to how its content is often dense mathematics.

From a software engineering perspective, the primary concern would be the use of abstract notation which, while logically might be familiar, are syntaxtically unorthodox. An example of this is $\sum$ which represents iterative accumulation, comparable that to a programmatical *for*

loop. Because of this notation, software and game developers would first need to translate the content from the paper into pseudo code.

3.2.2 Computational Physics

Computational literature such as *Computer Simulation in Physics and Engineering* (Steinhauser, 2012, a)re often based on the implementation of the theory established by foundational literature. Because of this, it is less mathematically dense, and more focal on the practical implementation of the functions and concepts introduced.

However, with this particular example, the argument could be made that it has the inverse problem than the foundational paper mentioned above, focusing heavily on the basics of computer science, such as explaining binary and arithmetic-in-code, rather than the actual implementation of the mathematics.

3.2.3 Developer Experience & Accessibility

The disparity between these two types of academic literature creates a gap between the the Developer Experience (**DX**), which can result in longer development time as considerable time may be required to 'reverse-engineer' the mathematics.

Furthermore, it should be noted that aims of the reseacher and developer may differ.

While the researcher may be showing the metholodgy used to achieve a specified result. The developer often times would also also aim for efficiency and scalablity in the algorithms used; something that the researcher would not have considered.

Chapter 4

Accessibility Metric System

To provide a basis of the subjective development experience, each source will be subjected to an analysis which tracks the frequency and severity of instances that the source is lacking in clarity and requires switching to other resources to understand, or add to, the context written.

4.0.1 Scoring Stystem

Each source will accumulate points depending on how far and how long I need to refer to other sources to understand the original.

Table 4.1: Complexity Severity Levels

Level	**Value**	**Description**	**Examples**
Minor	1	Lexical issue. A word, phrase, or term that may not be immediately understood, requiring brief additional research.	'Isospin', 'Alpha Decay'
Moderate	2	Notation barrier. Mathematical notation that is not explained, requiring research into determining its meaning.	$\sum$
Major	4	Unexplained introduced concepts. Concepts that are mentioned but not elaborated, which may require extensive further research.	—
Critical	8	Incomprehensive topic. A topic is difficult to research or understand in the time given; e.g., a proof with no numerical constants.	Complex proofs

4.0.2 Evaluation Process

As the particle simulation is developed, each time the workflow is interrupted with needing to find context through additional resources, points will be accumulated and logged for each major resource, and a final **Approachability Score** will be created using the following equation:

$$A = \sum (P_{\text{minor}} + P_{\text{moderate}} + P_{\text{major}} + P_{\text{critical}}) \tag{4.1}$$

A lower Approachability Score indicates a more developer-friendly resource. Whereas a higher score suggests a more difficult resource for developers to use.

Chapter 5

Methodology

The methodology of this project will be double-faceted, focusing on the development of the simulation itself, and the concurrent audit of sources used to aide that development.

5.1 Simulation Architecture

The simulation is developed using C++ due to its high performance and stability, and memory management which will be important for handling large particle counts and their calculations efficiently.

- **Object-orientated design:** Each particle will be considered it's own object. Handling it's own position, radius, and systems such as rendering.
- **Environment:** Development is performed in a Linux environment using the GCC compiler.

The simulation also uses Dear ImGui (commonly referred to as *ImGui, imgui, or IMGUI*) to render a Graphical User Interface (GUI), this will provide the user statistics such as the number of particles, performance stats, alongside providing the user the ability to add or remove particles. (omar, 2026)

5.2 Approachability Audit

5.2.1 Data Collection Process

As additions are made to the simulation (e.g the gravitational force), the primary source that was used for that feature will be logged, and its approachability will be scored and logged based on the criteria shown in table 4.1.

5.2.2 Evaluation

At the end of the development of the simulation, the **Approachability Score *AS*** will be calculated and cross-referenced to the estimated time it had taken to implement the feature the source was used for.

5.2.3 Problem with Units & Scales

The first issue encountered was the ubiquitous application of the International Systemm of Units **SI** (National Institute of Standards and Technology, 2010), which standardises the accepted use, measurement, and notation of quantaties, including mass, weight, speed, and distance.

The use of SI Units by scientific literature creates a problem when translating equations to code. This is because many values, such as the virtually infinitesimal mass of a proton: 1.67×10^{-27} kg become difficult to represent by a computer, often leaving to floating-point precision errors.

To avoid the imprecision of computational small numbers, we must normaliser the SI Units into a more manageable scale. By selecting a base reference, such as the mass of a proton, we can define a new coordinate system where all other values are represented relative to that base reference.

For the simulation, the mass of a Proton (m_p) was selected as the fundemental unit of measurement ($1.000su$). All other masses are converted by dividing their SI value by m_p.

Table 5.1: SI to SU Conversion Examples

Name	SI Unit (kg)	Equation	Converted Simulation Unit (su)
Proton	1.67×10^{-27}	$(1.67 \times 10^{-27}) \div (1.67 \times 10^{-27})$	1.000
Neutron	1.67×10^{-27}	$(1.67 \times 10^{-27}) \div (1.67 \times 10^{-27})$	1.000
Electron	9.11×10^{-31}	$(9.11 \times 10^{-31}) \div (1.67 \times 10^{-27})$	0.0005

5.3 Fundamental Forces

Now that the problem with units and scales has been addressed, the first major feature could be implemented, the Fundemental Forces. For this, I heavily relied on *A Modern Introduction to Particle Physics* (Fayyazuddin & Riazuddin, 2011, f)or the base of the mathematics and general understanding.

At this point the project boilerplate has been written, such as the imgui interface, and the ability to place either a *red* proton *(p)*, *blue* neutron *(n)*, *yellow* electron *(e^-)*, or a *purple* photon *(ph)*, into the sandbox scene.

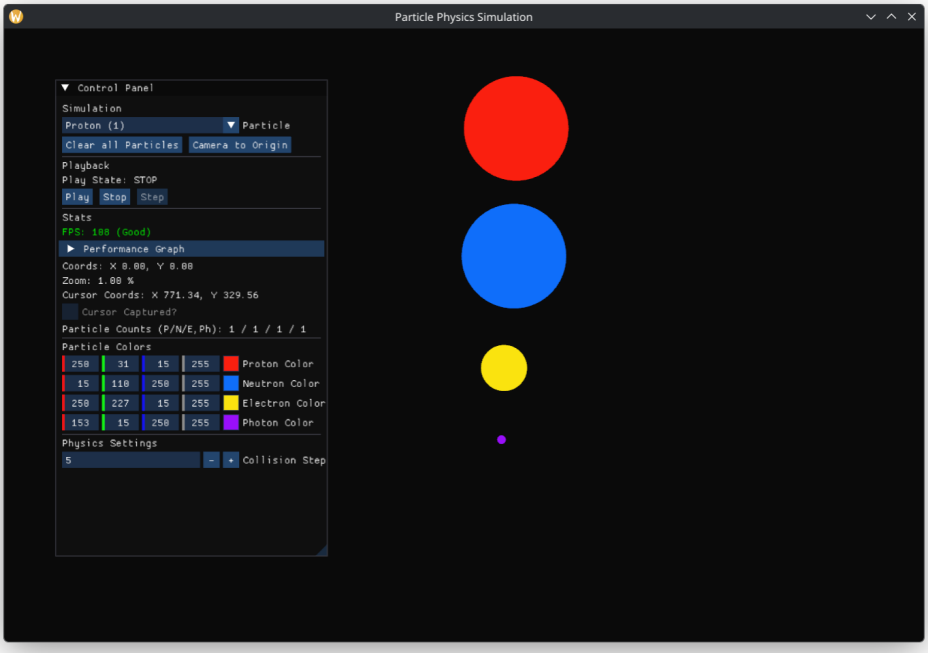

Figure 5.1: App window showing imgui interface and the particles

5.3.1 Gravitational

The gravitational force, stated by Fayyazuddin and Riazuddin (2011) (p. 2), is a 'force of attraction between two particles and is proportional to their gravitational charges, namely their masses". This definition was very easy to understand, stating that the force is the how attracted two particles are given their mass.

However, when the implementation of the force began, I quickly ran into the first **Lexical** issue. The paper noted that the strength of the gravitational force was:

$$G_n = 1.3 \times 10^{-54} (GeV/m)^{-1} \tag{5.1}$$

Fayyazuddin and Riazuddin, 2011, p. 2

The first half of this equation made sense: $G_n = 1.3 \times 10^{-54}$, but what was $(GeV/m)^{-1}$?

List of Figures

5.1 App window showing imgui interface and the particles 10

List of Tables

4.1 Complexity Severity Levels 6

5.1 *SI* to *SU* Conversion Examples 9

References

Fayyazuddin & Riazuddin. (2011). *A modern introduction to particle physics* (3rd ed.). World Scientific Publishing Company.

Moore, T. A. (A. (2024). *A standard model workbook*. University Science Books.

National Institute of Standards and Technology. (2010, April 12). *Si units* [Last modified 2025-07-22]. National Institute of Standards and Technology. Retrieved April 8, 2026, from https://www.nist.gov/pml/owm/metric-si/si-units

omar. (2026, April 3). *Ocornut/imgui*. Retrieved April 3, 2026, from https://github.com/ocornut/imgui

Parsons, T., & Bester, R. (2026, March 11). *Marmalade-Engine/marmalade*. Retrieved April 8, 2026, from https://github.com/Marmalade-Engine/marmalade

Steinhauser, M. O. (2012). *Computer Simulation in Physics and Engineering*. De Gruyter, Inc. Retrieved March 4, 2026, from http://ebookcentral.proquest.com/lib/portsmouth-ebooks/detail.action?docID=893213